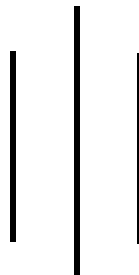


Tribhuvan University
Institute of Science and Technology



A Term Paper
On
“Evaluation and Context Optimization of Retrieval-Augmented Generation for
Document Question Answering”

Submitted To
Central Department of Computer Science and Information Technology
Tribhuvan University, Kirtipur
Kathmandu, Nepal



Submitted By
Melina Giri
Roll No: 8125018
April, 2026

In partial fulfillment of the requirement for Master’s Degree in Information
Technology (MIT), Third Semester



Tribhuvan University
Institute of Science and Technology

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this term paper, prepared under the supervision of Asst. Prof. Balkrishna Subedi entitled "**Evaluation and Context Optimization of Retrieval-Augmented Generation for Document Question Answering**" be accepted as fulfillment in partial requirement for the degree of Master's in Information Technology. To the best of my knowledge, this is an original work in computer science.

.....

Asst. Prof. Balkrishna Subedi

(Supervisor)

Central Department of Computer Science and Information Technology

LETTER OF APPROVAL

This is to certify that the seminar report prepared by **Ms. Melina Giri** entitled **"Evaluation and Context Optimization of Retrieval-Augmented Generation for Document Question Answering"** in partial fulfillment of the requirements for the degree of Masters in Information Technology has been well studied. In our opinion, it is satisfactory in the scope and quality as a project for the required degree.

Evaluation Committee

.....
Asst. Prof. Dr. Dhiraj Kedar Pandey
(H.O.D)
Central Department of Computer Science and
Information Technology

.....
Asst. Prof. Balkrishna Subedi
(Supervisor)
Central Department of Computer Science and
Information Technology

.....
(Internal)

ACKNOWLEDGEMENT

The success and final outcome of this report required a lot of guidance and assistance from many people and I am very fortunate to have got this all along the completion. I am very glad to express my deepest sense of gratitude and sincere thanks to my highly respected and esteemed supervisor **Asst. Prof. Balkrishna Subedi**, Central Department of Computer Science and Information Technology for his valuable supervision, guidance, encouragement, and support for completing this paper.

I am also thankful to **Asst. Prof. Dr. Dhiraj Kedar Pandey**, HOD of Central Department of Computer Science and Information Technology for his constant support throughout the period. Furthermore, with immense pleasure, I submit my deepest gratitude to the Department of Computer Science and Information Technology, Tribhuvan University, and all the faculty members of CDCSIT for providing the platform to explore the knowledge of interest.

I would also like to acknowledge the authors and contributors of the **Open RAG Benchmark** provided by Vectara. The publicly available benchmark dataset was fundamental for conducting the retrieval and generation experiments presented in this research.

At the end I would like to express my sincere thanks to all my friends and others who helped me directly or indirectly.

Melina Giri (8125018)

Central Department of Computer Science and Information Technology

ABSTRACT

Retrieval-Augmented Generation (RAG) combines information retrieval with language generation to provide answers using relevant information retrieved from external document collections. This study implements and evaluates a **RAG-based document question-answering system** using the **Vectara Open RAG Benchmark**. The system processes PDF documents, divides extracted text into smaller sections, generates dense embeddings using the **all-MiniLM-L12-v2** model, and retrieves relevant document sections using vector similarity. Retrieval performance was evaluated using **3,045 benchmark queries** at retrieval depths of $k=3, 5, 10$, and 20 using Hit@ k , Mean Reciprocal Rank (MRR), and normalized Discounted Cumulative Gain (nDCG). The results show that retrieval effectiveness improved with increasing retrieval depth, with Hit@20 reaching **82.33%**, compared with **61.18%** at $k=3$. Generation performance was evaluated using **60 questions** under three configurations: **R0_k3**, **R1_k10**, and **R2_k10_optimized**. The final generation experiment used the locally executed **Qwen2.5-3B** language model. Among the three configurations, R1_k10 achieved the highest mean lexical F1 score of **0.3136**, while R2_k10_optimized achieved the lowest average generation latency of **12.098 seconds**. The findings indicate that increasing retrieval depth improves retrieval coverage, while context optimization can improve generation efficiency but does not necessarily improve lexical answer quality. The study also identifies the limitations of lexical F1 and recommends semantic and human-based evaluation methods for future research.

Keywords: Retrieval-Augmented Generation, Document Question Answering, Dense Retrieval, Sentence Embeddings, Vector Database, Context Optimization, Open RAG Benchmark, Qwen2.5.

Table of Contents

SUPERVISOR’S RECOMMENDATION	i
LETTER OF APPROVAL	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
Table of Contents	v
LIST OF FIGURES	vii
LIST OF TABLES	viii
LIST OF ABBREVIATIONS	ix
CHAPTER 1: INTRODUCTION	1
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Objectives	2
CHAPTER 2: PREVIOUS WORKS, DISCUSSIONS AND FINDINGS	4
2.1 Background Study	4
2.2 Literature Review	4
2.3 Methodology	5
2.4 Implementation	10
2.5 Result Analysis	11
CHAPTER 3: CONCLUSION	17
3.1 Conclusion	17
3.2 Future Recommendation	17

References	20
Appendix	21
A.1 Main Interface Code	21
A.2 Loading Documents and Processing Text	22
A.3 Embedding Generation and Vector Retrieval	23
A.4 RAG Answer Generation Code	23
A.5 Evaluation and Result Analysis Code	24

LIST OF FIGURES

Figure 1 : Overview of RAG-Based Document Question-Answering Workflow	6
Figure 2 : Retrieval Performance at Different Retrieval Depths	12
Figure 3 : Lexical F1 Comparison of RAG Generation Configurations	13
Figure 4 : Average Generation Latency Across RAG Configurations	13
Figure 5 : Sample Question and Generated Answer from the RAG System	15
Figure 6 : RAG pipeline	15

LIST OF TABLES

Table 1 : Retrieval Performance at Different Retrieval Depths	11
Table 2 : Generation Performance of Different RAG Configurations	14
Table 3 : Paired Lexical-F1 Differences	14

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
F1	F1 Score
LLM	Large Language Model
MRR	Mean Reciprocal Rank
nDCG	normalized Discounted Cumulative Gain
PDF	Portable Document Format
QA	Question Answering
RAG	Retrieval-Augmented Generation
IDK	I Don't Know
F	FastAPI
k	Retrieval Depth / Number of Retrieved Sections

CHAPTER 1: INTRODUCTION

1.1 Introduction

The rapid growth of digital documents and research publications has created a significant challenge in efficiently finding and extracting relevant information. Traditional document search systems generally rely on keyword matching, which may return large numbers of documents without providing a direct, context-aware answer to the user's question. As the volume of digital information continues to increase, there is a growing need for systems that can retrieve relevant information and generate meaningful answers from that information.

Recent advances in **Large Language Models (LLMs)** have significantly improved natural language question-answering capabilities. However, language models may have limitations when answering knowledge-intensive questions that require information from external document collections. This limitation has motivated the development of **Retrieval-Augmented Generation (RAG)** systems [1].

RAG combines **information retrieval** with **language generation**. Instead of relying only on information encoded in the language model, a RAG system first retrieves relevant sections from an external document collection and provides those sections as context to the language model. The model subsequently generates an answer using the retrieved context [1]. A typical RAG pipeline consists of document processing, text chunking, embedding generation, vector-based retrieval, context selection, and answer generation [1].

The effectiveness of a RAG system depends not only on the quality of the language model but also on the quality and quantity of retrieved context. Retrieving too few sections may omit information necessary to answer a question, whereas retrieving too many sections may introduce redundant or irrelevant information and increase computational cost. Therefore, determining an appropriate retrieval depth and optimizing the retrieved context are important considerations in RAG-based document question answering.

This study evaluates a RAG system for document question answering using the **Open RAG Benchmark** [4], focusing on retrieval performance and the effect of different retrieval and context-optimization strategies. The study evaluates retrieval at different values of k and compares three generation conditions: retrieval of the top three sections, retrieval of the top ten sections, and retrieval of the top ten sections followed by context optimization. The study

therefore investigates the relationship between retrieval depth, context optimization, answer quality, and generation latency.

1.2 Problem Statement

Although Retrieval-Augmented Generation systems can improve document-based question answering by supplying relevant external context to a language model, their performance is strongly influenced by the retrieval strategy and the amount of context provided to the model. Retrieving a small number of sections may fail to provide sufficient information for answering a question, while retrieving a larger number of sections may introduce redundant information and increase processing requirements.

Furthermore, simply increasing the number of retrieved sections does not necessarily guarantee better generated answers. Context selection and optimization may reduce unnecessary information and generation latency, but excessive removal of context may also eliminate information required for producing an accurate answer.

Therefore, there is a need to experimentally evaluate how **retrieval depth and context optimization affect RAG performance**. This study addresses this problem by evaluating retrieval effectiveness at different k values and comparing multiple generation configurations using a standardized document question-answering benchmark.

1.3 Objectives

To evaluate the performance of a Retrieval-Augmented Generation (RAG) system for document question answering and analyze the effects of retrieval depth and context optimization on answer quality and generation efficiency.

The specific objectives of this study are:

- **To implement a document question-answering pipeline using Retrieval-Augmented Generation**, incorporating document processing, text chunking, embedding generation, vector-based retrieval, and language-model-based answer generation.
- **To evaluate retrieval performance at different retrieval depths** using Hit@ k , Mean Reciprocal Rank (MRR), and nDCG metrics.
- **To compare different RAG generation configurations**, including top-3 retrieval, top-10 retrieval, and optimized top-10 retrieval.
- **To evaluate generated answers using lexical F1** as an automatic answer-overlap metric.

- **To analyze the effect of context optimization on generation efficiency**, particularly in terms of context size and generation latency.
- **To identify the strengths and limitations of different retrieval and context-selection strategies** and provide recommendations for improving RAG-based document question-answering systems.

CHAPTER 2: PREVIOUS WORKS, DISCUSSIONS AND FINDINGS

2.1 Background Study

The rapid growth of digital documents and research publications has created a significant challenge in efficiently finding and extracting relevant information. Traditional document retrieval systems mainly depend on keyword matching, which may require users to examine multiple documents before finding the information they need. As the volume of digital information continues to increase, effective document question-answering systems have become increasingly important.

Early document question-answering approaches were largely based on **keyword search, rule-based methods, and information retrieval techniques**. Although these approaches can identify documents containing specific terms, they may have difficulty understanding the semantic meaning of a question or producing a direct answer from multiple relevant passages.

The development of **Large Language Models (LLMs)** has significantly improved natural-language question answering. However, language models may have limitations when answers require information from specific external document collections. This limitation has led to the development of **Retrieval-Augmented Generation (RAG)** systems [1].

RAG combines **information retrieval and language generation**. Relevant information is first retrieved from an external document collection and then supplied as context to a language model. The model uses this retrieved context to generate an answer to the user's question [1]. A typical RAG system includes document processing, text chunking, embedding generation, vector storage, similarity-based retrieval, context selection, and answer generation.

The performance of a RAG system depends strongly on the quality and quantity of retrieved context. Retrieving too few sections may omit important information, while retrieving too many sections may introduce redundant information and increase computational requirements. Therefore, retrieval depth and context optimization are important factors in developing efficient document question-answering systems.

2.2 Literature Review

The increasing availability of digital documents has made efficient document retrieval and question answering an important research area. Traditional information retrieval methods

generally rely on lexical matching between queries and documents. Although effective for exact keyword searches, such methods may fail when the query and relevant document use different words with similar meanings.

The introduction of **dense vector representations and embeddings** has provided a semantic approach to information retrieval. Instead of matching only individual keywords, text can be converted into numerical vectors, allowing semantically similar queries and document sections to be identified using similarity measures [2][3]. **Cosine similarity** can then be used to measure the similarity between vector representations.

Recent developments in **Large Language Models** have further improved natural-language generation. However, LLM-only systems may suffer from hallucination, outdated knowledge, and lack of access to domain-specific documents. RAG addresses these limitations by retrieving external evidence before generating an answer. This approach provides the language model with retrieved document context during answer generation [1].

An important component of RAG performance is the **number of retrieved documents or sections**, commonly represented by k . A small k may provide insufficient context, whereas a large k may introduce irrelevant or repetitive information. Context optimization techniques can therefore be applied to remove redundant sections before generation.

RAG systems are commonly evaluated at two levels: **retrieval performance** and **generation performance**. Retrieval can be evaluated using measures such as **Hit@k**, **Mean Reciprocal Rank (MRR)**, and **normalized Discounted Cumulative Gain (nDCG)**. Generated answers can be evaluated using automatic metrics such as lexical F1, although lexical measures primarily evaluate word overlap and should not be interpreted as complete measures of semantic correctness.

In conclusion, previous research establishes that effective RAG performance depends on both **retrieval quality and generation context**. This foundation supports the methodology adopted in the present study, which evaluates different retrieval depths and compares context configurations to investigate their effects on answer quality and generation efficiency.

2.3 Methodology

2.3.1 Flowchart

The overall workflow of the proposed RAG-based document question-answering system consists

of document preparation, text processing, embedding generation, vector retrieval, context selection, and answer generation.

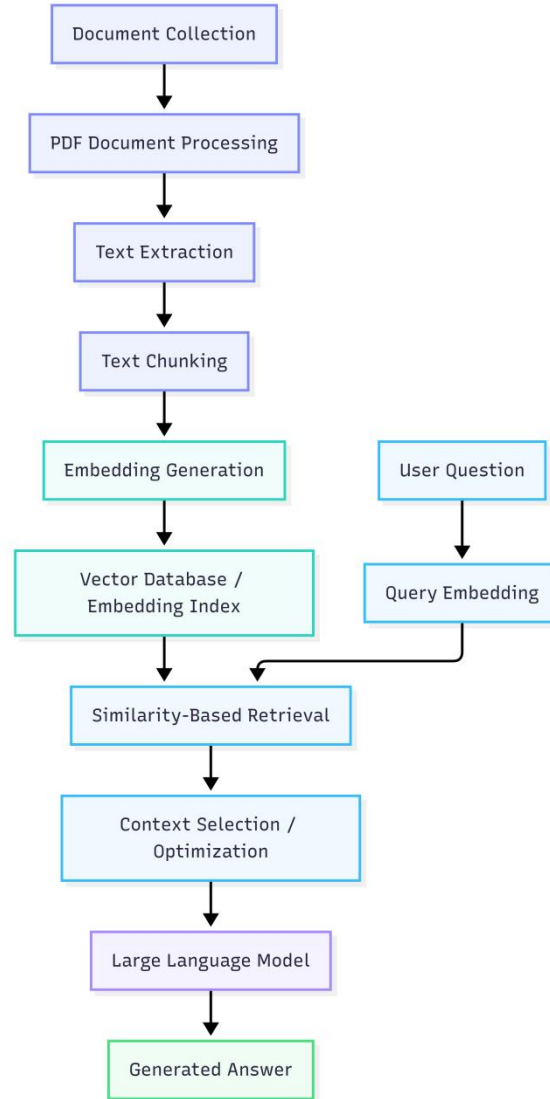


Figure 1: Overview of RAG-Based Document Question-Answering Workflow

2.3.2 Dataset Description

The dataset used for this study is the **Vectara Open RAG Benchmark**, which is designed for evaluating retrieval-augmented generation systems on research-paper documents. The benchmark contains **1,000 PDF papers** and **3,045 valid question-answer pairs**. The benchmark includes both abstractive and extractive questions and contains text, text-image, text-table, and text-table-image source categories [4].

Data Processing

Before performing retrieval, the document corpus is processed and transformed into searchable text sections and numerical vector representations. The main processing steps include:

1. Document Preprocessing

- 1.1 **PDF processing:** Research papers were processed to obtain their textual content and document sections.
- 1.2 **Text chunking:** Extracted text was divided into smaller sections to make retrieval more precise.
- 1.3 **Metadata preparation:** Document identifiers and section identifiers were retained to compare retrieved sections with the benchmark's relevant sections.
- 1.4 **Embedding generation:** Each document section was converted into a numerical vector representation.

2. Embedding Generation

The study uses the **all-MiniLM-L12-v2** sentence-transformer model to generate dense embeddings for document sections and queries [3][6].

Each document section is represented as a **384-dimensional vector**. These vectors allow semantic similarity to be calculated between a user query and **candidate document sections** [6].

Retrieval Analysis

The retrieval stage uses dense-vector similarity to identify the most relevant document sections for a given question, following the general principle of dense retrieval [2]. The query is converted into the same embedding space as the document sections, and similarity scores are calculated.

Different retrieval depths were evaluated:

- **k = 3**
- **k = 5**
- **k = 10**
- **k = 20**

The retrieved sections were then compared against the benchmark's relevant sections.

Cosine Similarity (Mathematical Formula)

The similarity between a query vector $\mathbf{q}$ and document vector $\mathbf{d}$ is calculated using:

$$\text{Similarity}(q, d) = \frac{q \cdot d}{\|\mathbf{q}\| \cdot \|\mathbf{d}\|}$$

Where:

- $\mathbf{q}$ = query embedding vector
- $\mathbf{d}$ = document-section embedding vector
- $\mathbf{q} \cdot \mathbf{d}$ = dot product between the two vectors
- $\|\mathbf{q}\|$ = magnitude of query vector
- $\|\mathbf{d}\|$ = magnitude of document vector

Retrieval Evaluation Metrics

The retrieval system was evaluated using:

Hit@k: Measures whether at least one relevant document section appears within the top k retrieved results.

Mean Reciprocal Rank (MRR): Measures how highly the first relevant result is ranked.

nDCG: Measures ranking quality while giving greater importance to highly ranked relevant results.

2.3.3 Algorithm Description

The algorithm aims to answer document-based questions using **Retrieval-Augmented Generation (RAG)** [1]. It involves preprocessing documents, generating embeddings, retrieving relevant sections, optionally optimizing the retrieved context, and generating an answer using a language model.

Algorithm 1: Retrieval-Augmented Generation for Document Question Answering

Input: Document corpus D , user question q , retrieval depth k

Output: Generated answer A

Steps:

1. Document Processing

- 1.1 Load the document corpus.
- 1.2 Extract textual content from PDF documents.

- 1.3 Divide documents into manageable sections.
- 1.4 Generate embeddings for each document section.

2. Query Processing

- 2.1 Accept the question from the user.
- 2.2 Generate an embedding representation of the question.
- 2.3 Retrieve Relevant Sections
- 2.4 Calculate similarity between the query embedding and document-section embeddings.
- 2.5 Rank document sections according to similarity.
- 2.6 Select the top-k sections.

3. Context Optimization

For the optimized configuration:

- 3.1 Examine the retrieved sections.
- 3.2 Identify highly redundant sections using text overlap.
- 3.3 Remove unnecessary or repetitive sections.
- 3.4 Construct a smaller context for the language model.

4. Generate Answer

- 4.1 Combine the question with the retrieved context.
- 4.2 Provide the context and question to the language model.
- 4.3 Generate the final answer.
- 4.4 If the available context is insufficient, the system is instructed to respond that the information is unknown.

2.4 Implementation

2.4.1 Implementation Tools

The implementation was carried out using the tools below for document processing, retrieval, vector representation, and answer generation:

- **Python 3.x** – Main programming language
- **FastAPI** – Backend API framework
- **Streamlit** – User-interface/frontend framework
- **LangChain** – RAG pipeline and language-model integration
- **Sentence Transformers** – Generation of dense text embeddings
- **ChromaDB / vector indexing** – Vector storage and retrieval
- **HuggingFace** – Embedding model
- **Ollama** – Local language-model execution
- **Qwen2.5 3B** – Local language model used for final generation experiment
- **NumPy** – Numerical computation and embedding operations
- **Pandas** – Dataset processing and result analysis
- **Matplotlib** – Visualization of experimental results
- **VS Code** – Development environment
- **Dataset:** Vectara Open RAG Benchmark

2.4.2 Implementation Details

The RAG system converts document sections and user queries into dense numerical representations using the all-MiniLM-L12-v2 embedding model. Each representation contains 384 dimensions. Query and document vectors are compared using cosine similarity to identify the most relevant document sections.

The retrieval experiment was performed using 3,045 benchmark queries. Retrieval performance was evaluated at four different retrieval depths: $k=3$, 5, 10, and 20.

For generation evaluation, 60 questions were selected from the benchmark. Three retrieval-context configurations were evaluated:

- **R0_k3:** Top 3 retrieved sections were provided to the language model.
- **R1_k10:** Top 10 retrieved sections were provided to the language model.
- **R2_k10_optimized:** Top 10 sections were retrieved and redundant context was

removed before generation.

The final generation experiment used the **Qwen2.5-3B** language model running locally through Ollama [5]. Local execution eliminated dependence on an external API daily-token quota during the final experiment.

The generated answers were compared with the benchmark reference answers using lexical F1. Generation latency and context size were also recorded to evaluate efficiency.

2.5 Result Analysis

2.5.1 Quantitative Analysis

Retrieval Performance at Different k Values

Table 1: Retrieval Performance at Different Retrieval Depths

Retrieval Depth	Hit@k	MRR	nDCG
k = 3	0.6118	0.5061	0.5333
k = 5	0.6893	0.5239	0.5653
k = 10	0.7718	0.5349	0.5920
k = 20	0.8233	0.5385	0.6051

The retrieval results demonstrate that increasing the retrieval depth consistently improved retrieval performance.

- At **k=3**, Hit@3 was **61.18%**.
- At **k=5**, Hit@5 increased to **68.93%**.
- At **k=10**, Hit@10 increased to **77.18%**.
- At **k=20**, Hit@20 reached **82.33%**.

Similarly, MRR increased from **0.5061** at **k=3** to **0.5385** at **k=20**, while nDCG increased from **0.5333** to **0.6051**.

These results indicate that increasing retrieval depth improves the probability of retrieving relevant information and also improves the overall ranking quality.

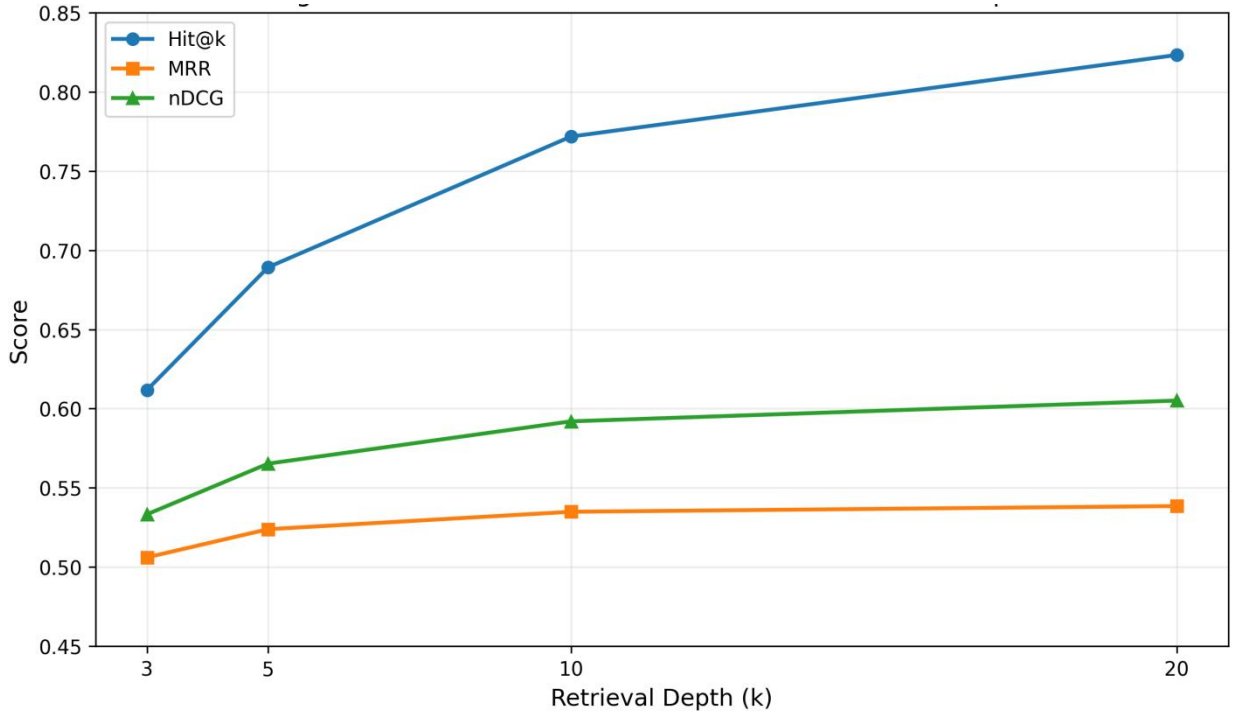


Figure 2: Retrieval Performance at Different Retrieval Depths

Generation Latency Analysis

The generation latency showed a substantial difference between the three configurations.

- **R0_k3:** 185.600 seconds
- **R1_k10:** 16.958 seconds
- **R2_k10_optimized:** 12.098 seconds

The optimized configuration achieved the lowest average latency. Compared with R1_k10, R2_k10_optimized reduced average latency by approximately **4.86 seconds**. The substantially higher latency observed for R0_k3 indicates that generation latency was affected by the local inference environment and runtime behavior in addition to the amount of retrieved context.

This demonstrates that reducing unnecessary context can improve generation efficiency.

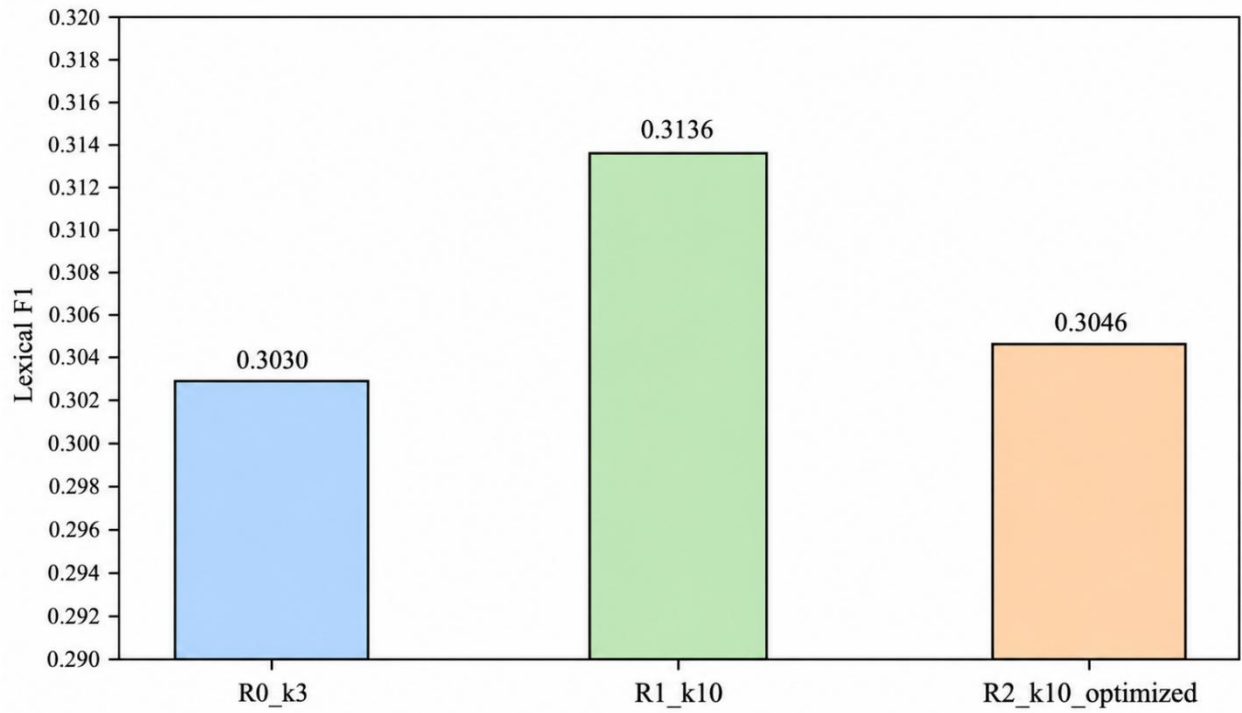


Figure 3: Lexical F1 Comparison of RAG Generation Configurations

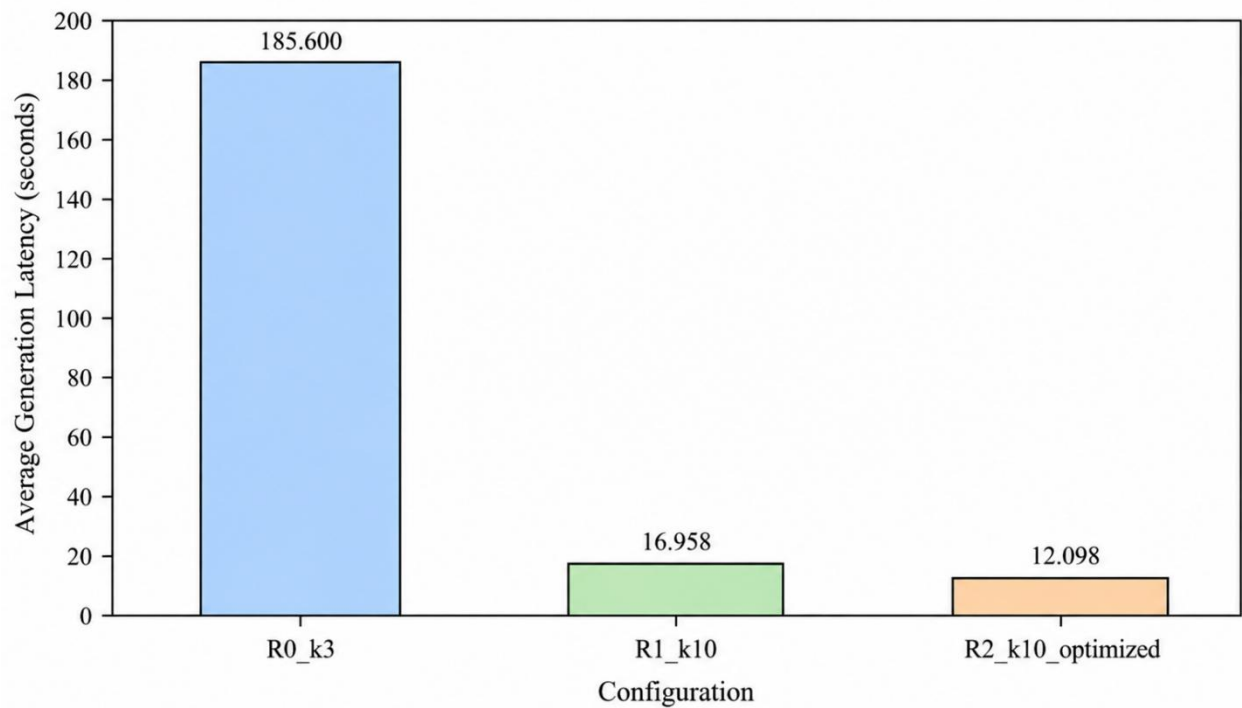


Figure 4: Average Generation Latency Across RAG Configurations

Table 2: Generation Performance of Different RAG Configurations

Metric	R0_k3	R1_k10	R2_k10_optimized
Questions	60	60	60
Successful	60	60	60
Errors	0	0	0
Mean Lexical F1	0.3030	0.3136	0.3046
IDK	23.33%	23.33%	25.00%
Mean Latency (s)	185.600	16.958	12.098
Mean Context Sections	2.45	3.23	3.10
Mean Context Characters	7014	8000	7862

Paired Lexical-F1 Difference

Table 3: Paired Lexical-F1 Differences

Comparison	Change in Lexical F1
R0_k3 → R1_k10	+0.0106
R1_k10 → R2_k10_optimized	−0.0090
R0_k3 → R2_k10_optimized	+0.0016

The comparison shows that increasing retrieval depth from 3 to 10 sections produced the largest positive change in lexical F1. Context optimization subsequently produced a small reduction.

Overall, the optimized configuration remained slightly above the baseline R0 configuration, with a net improvement of **0.0016**.

2.5.2 Qualitative Analysis (Actual System Output)

The implemented RAG system accepts document-based questions and retrieves relevant sections before generating an answer. The final generation experiment was performed using the locally hosted **Qwen2.5 3B** model.

A sample output from the system can be presented in this section to demonstrate the complete question-answering process.

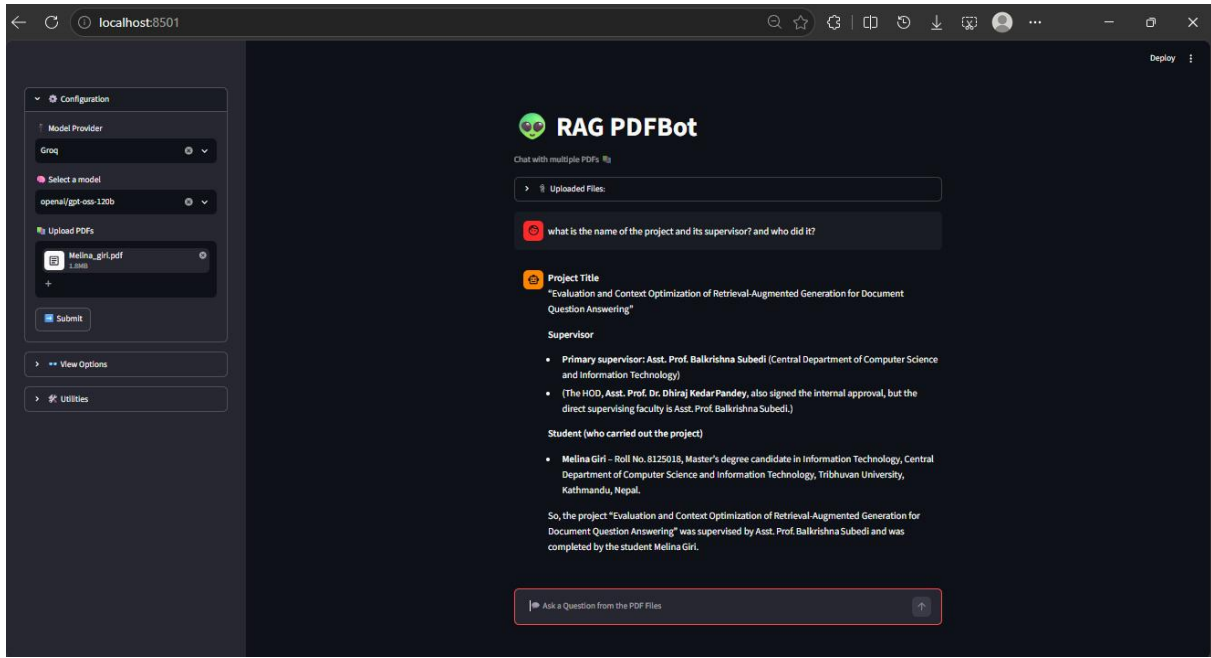


Figure 5: Sample Question and Generated Answer from the RAG System

The qualitative output demonstrates the complete RAG pipeline:



Figure 6: RAG pipeline

The generated response is produced using the retrieved document context supplied to the language model as part of the RAG pipeline [1].

Analysis of the Three Configurations

R0_k3:

The system uses only the top three retrieved sections. This provides a relatively small context but may omit information required for answering complex questions.

R1_k10:

The system retrieves ten sections, providing more information to the language model. This configuration achieved the highest lexical F1 score (**0.3136**) among the three generation configurations.

R2_k10_optimized:

The system initially retrieves ten sections and then removes redundant information. Although its

lexical F1 (**0.3046**) was slightly lower than R1, it achieved the lowest generation latency (**12.098 seconds**), demonstrating an efficiency advantage.

Overall, the experimental results suggest that **retrieval depth has a positive effect on retrieval performance**, while **context optimization can improve generation efficiency but does not necessarily improve answer quality measured through lexical F1**. The lexical F1 results should be interpreted as an automatic word-overlap measure rather than definitive semantic correctness.

CHAPTER 3: CONCLUSION

3.1 Conclusion

This research successfully implemented and evaluated a Retrieval-Augmented Generation (RAG) system for document question answering based on the retrieval-and-generation framework described in previous RAG research [1]. The study demonstrated how combining semantic information retrieval with a language model can provide answers based on relevant sections retrieved from a document collection rather than relying solely on the language model's internal knowledge.

Through a systematic methodology involving PDF document processing, text chunking, embedding generation, vector-based retrieval, context selection, and answer generation, the system was able to retrieve relevant document sections and use them as context for generating answers. The retrieval component was evaluated using different retrieval depths, including $k=3$, $k=5$, $k=10$, and $k=20$.

The retrieval results showed that increasing the number of retrieved sections generally improved retrieval effectiveness. Hit@ k increased from 61.18% at $k=3$ to 82.33% at $k=20$, while MRR increased from 0.5061 to 0.5385. Similarly, nDCG increased from 0.5333 to 0.6051, indicating improved ranking quality as retrieval depth increased.

The generation experiment compared three configurations: R0_k3, R1_k10, and R2_k10_optimized. Among these configurations, R1_k10 achieved the highest mean lexical F1 score of 0.3136, compared with 0.3030 for R0_k3 and 0.3046 for R2_k10_optimized. Context optimization slightly reduced lexical F1 but improved generation efficiency, reducing average latency from 16.958 seconds to 12.098 seconds.

The final experiment successfully generated answers for 60 questions under all three configurations, resulting in 180 successful generations without errors. The results demonstrate that increasing retrieval depth can improve the availability of relevant information, while context optimization can reduce unnecessary context and generation time.

In conclusion, the study validates that retrieval depth and context selection are important factors in RAG-based document question answering. The findings show that providing additional relevant context can improve retrieval and slightly improve lexical answer overlap, while context optimization can provide efficiency benefits. However, lexical F1 should be interpreted as an automatic word-overlap measure rather than definitive semantic answer correctness.

3.2 Future Recommendation

1. **Expand the Generation Evaluation Dataset**

The current generation experiment evaluates 60 questions from the benchmark, while the retrieval evaluation uses the complete set of 3,045 queries. Future work can evaluate generation on a larger number of questions to obtain more statistically robust results and reduce the effect of sample selection.

2. **Improve Context Optimization**

The current optimization method removes sections based on textual redundancy. Future work can investigate more advanced context-selection methods based on semantic similarity, relevance scores, diversity, and learned ranking techniques. This may reduce unnecessary context while preserving information required for accurate answers.

3. **Evaluate Different Embedding Models**

The current system uses all-MiniLM-L12-v2 for document and query embeddings. Future studies can compare other embedding models to determine whether stronger semantic representations improve retrieval performance, particularly for complex and domain-specific questions.

4. **Evaluate Different Language Models**

The final experiment uses the locally executed Qwen2.5 3B model. Future research can compare different open-weight and hosted language models to determine how model capability affects answer quality, hallucination, latency, and resource requirements.

5. **Use Semantic Answer Evaluation**

The current generation evaluation primarily uses lexical F1, which measures word overlap between generated and reference answers. Future work should incorporate semantic evaluation methods such as BERTScore, embedding-based similarity, LLM-as-a-judge evaluation, and human evaluation to better measure whether generated answers are actually correct.

6. **Improve Retrieval and Ranking**

Although increasing k improved retrieval coverage, retrieving additional sections may also introduce irrelevant information. Future work can investigate reranking models, hybrid keyword-plus-dense retrieval, and query expansion to improve the relevance of retrieved context.

7. Integration into Real Applications

Beyond benchmark evaluation, the RAG system can be deployed as a practical document question-answering application for research papers, educational materials, organizational documents, or other specialized document collections. Evaluating response time, user satisfaction, answer correctness, and resource consumption in real-world environments would provide further practical validation.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Y. Wang, A. Piktus, et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [2] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense Passage Retrieval for Open-Domain Question Answering,” *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781, 2020.
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.
- [4] Vectara, “Open RAG Benchmark,” Vectara Research and Benchmark Dataset.
- [5] Qwen Team, “Qwen2.5: A Party of Foundation Models,” 2024. The Qwen2.5-3B model card identifies the model and provides its citation information.
- [6] Sentence-Transformers, “all-MiniLM-L12-v2,” Hugging Face Model Card..

Appendix

A.1 Main Interface Code

This section shows the main interface of the RAG-based document question-answering system. The interface allows the user to provide a question and receive an answer generated from relevant retrieved document sections.

```
client > app.py > ...
1 import streamlit as st
2
3 from state.session import setup_session_state, is_chat_ready
4 from components.chat import (
5     render_chat_history,
6     render_download_chat_history,
7     render_uploaded_files_expander,
8     render_user_input
9 )
10 from components.sidebar import (
11     render_model_selector,
12     render_view_selector,
13     sidebar_file_upload,
14     sidebar_provider_change_check,
15     sidebar_utilities
16 )
17 from components.inspector import render_inspect_query
18
19
20 def main():
21     st.set_page_config(page_title="RAG PDFBot", layout="centered")
22     st.title("🤖 RAG PDFBot")
23     st.caption("Chat with multiple PDFs :books:")
24
25     setup_session_state()
26
27     if st.session_state.get("chat_history"):
28         render_download_chat_history()
29
30     with st.sidebar:
31         with st.expander("⚙️ Configuration", expanded=True):
32             model_provider, model = render_model_selector()
33             sidebar_file_upload(model_provider)
34             sidebar_provider_change_check(model_provider, model)
35
36         view_option = render_view_selector()
37         sidebar_utilities()
38
39     if not st.session_state.get(f"uploaded_files_{st.session_state.uploader_key}", []):
40         st.info("📄 Please upload and submit PDFs to start chatting.")
41
42     if st.session_state.get("unsubmitted_files", False):
43         st.warning("📄 New PDFs uploaded. Please submit before chatting.")
44
45     if st.session_state.get("chat_ready") and st.session_state.get("pdf_files", []):
46         render_uploaded_files_expander()
47
48     if view_option == "💬 Chat":
49         if st.session_state.get("chat_history", []):
50             render_chat_history()
51
52         if is_chat_ready():
53             render_user_input(model_provider, model)
54     elif view_option == "🔍 Inspector":
55         if is_chat_ready():
56             render_inspect_query(model_provider)
57
58 if __name__ == "__main__":
59     main()
```

Figure A. 1: Main Interface Code

A.2 Loading Documents and Processing Text

This section demonstrates how PDF documents are loaded and converted into textual sections before being used for embedding generation and retrieval.

The document-processing pipeline includes:

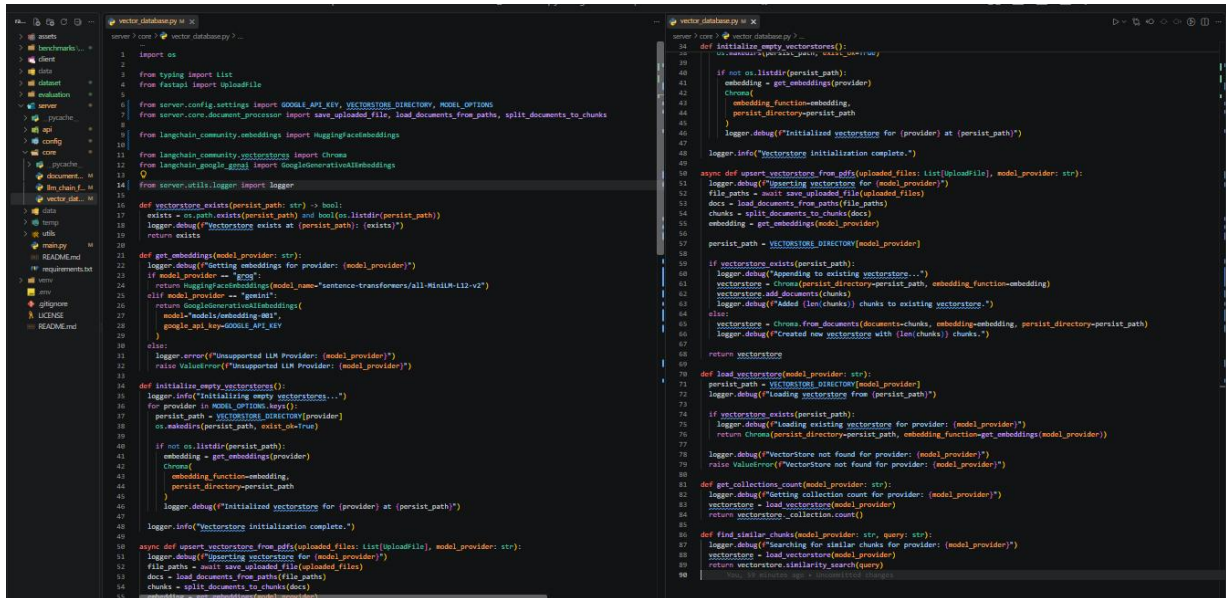
- Loading PDF documents
- Extracting text
- Splitting text into smaller sections
- Preparing document metadata
- Generating embeddings

```
document_processing.py | X
server > core > document_processing.py 3 ...
    You, 54 minutes ago | 2 authors (1 last edit and one other)
1  import os
2  import aiofiles
3
4  from typing import List
5  from fastapi import UploadFile
6
7  from langchain_community.document_loaders import PyPDFLoader
8  from langchain.text_splitters import TokenTextSplitter
9
10 from server.config.settings import TEMPFILE_UPLOAD_DIRECTORY
11 from server.utils.logger import logger
12
13 def validate_pdf(file: UploadFile, max_size_mb: int = 200):
14     if not file.filename.endswith(".pdf"):
15         logger.warning(f"Invalid file type: {file.filename}")
16         raise ValueError(f"{file.filename} is not a valid PDF file.")
17
18     file_size_mb = len(file.file.read()) / (1024 * 1024)
19     file.file.seek(0)
20
21     if file_size_mb > max_size_mb:
22         logger.warning(f"File too large: {file.filename} ({file_size_mb:.2f} MB)")
23         raise ValueError(f"PDF file size exceeds the maximum allowed size of {max_size_mb} MB.")
24
25     logger.debug(f"Validated PDF: {file.filename} ({file_size_mb:.2f} MB)")
26
27 async def save_uploaded_file(files: List[UploadFile]) -> List[str]:
28     os.makedirs(TEMPFILE_UPLOAD_DIRECTORY, exist_ok=True)
29     file_paths = []
30
31     for file in files:
32         validate_pdf(file)
33         file_path = os.path.join(TEMPFILE_UPLOAD_DIRECTORY, file.filename)
34         async with aiofiles.open(file_path, "wb") as f:
35             content = await file.read()
36             await f.write(content)
37         file_paths.append(file_path)
38         logger.debug(f"Saved uploaded file: {file.filename} to {file_path}")
39
40     return file_paths
41
42 def load_documents_from_paths(file_paths: List[str]):
43     docs = []
44     for file_path in file_paths:
45         loader = PyPDFLoader(file_path)
46         loaded = loader.load()
47         logger.debug(f"Loaded {len(loaded)} documents from {file_path}")
48         docs.extend(loaded)
49
50     return docs
51
52 def split_documents_to_chunks(docs) -> List[str]:
53     text_splitter = TokenTextSplitter(chunk_size=500, chunk_overlap=50)
54     chunks = text_splitter.split_documents(docs)
55     logger.debug(f"Split {len(docs)} docs into {len(chunks)} chunks")
56     return chunks
57
    You, 54 minutes ago • Uncommitted changes
```

Figure A. 2PDF Document Processing and Text Chunking Code

A.3 Embedding Generation and Vector Retrieval

The following code implements the generation of dense vector representations using the **all-MiniLM-L12-v2** embedding model [6]. The generated vectors are used to calculate similarity between the query and document sections.



```
vector_database.py
1 import os
2
3 from typing import List
4 from fastapi import UploadFile
5
6 from server.config.settings import MODEL_API_KEY, VECTORS_STORE_DIRECTORY, MODEL_PATH
7 from server.core.document_processor import save_uploaded_file, load_documents_from_paths, split_documents_to_chunks
8
9 from langchain_community.embeddings import HuggingFaceEmbeddings
10 from langchain_community.vectorstores import Chroma
11 from langchain_google_gemini import GoogleGenerativeAIEmbeddings
12
13 from server.utils.logger import logger
14
15 vector_db = None
16
17 def vectorstore_exists(persist_path: str) -> bool:
18     exists = os.path.exists(persist_path) and bool(os.listdir(persist_path))
19     logger.debug(f"Vectorstore exists at {persist_path}: {exists}")
20     return exists
21
22 def get_embeddings(model_provider: str):
23     logger.debug(f"Getting embeddings for provider: {model_provider}")
24     if model_provider == "gpt":
25         return HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L12-v2")
26     elif model_provider == "gemini":
27         return GoogleGenerativeAIEmbeddings(
28             model="models/embedding-001",
29             google_api_key=MODEL_API_KEY
30         )
31     else:
32         logger.error(f"Unsupported LLM Provider: {model_provider}")
33         raise ValueError(f"Unsupported LLM Provider: {model_provider}")
34
35 def initialize_empty_vectorstore():
36     logger.info(f"Initializing empty vectorstore...")
37     for provider in MODEL_PROVIDERS.keys():
38         persist_path = VECTORS_STORE_DIRECTORY[provider]
39         os.makedirs(persist_path, exist_ok=True)
40
41         if not os.listdir(persist_path):
42             embedding = get_embeddings(provider)
43             Chroma(
44                 embedding_function=embedding,
45                 persist_directory=persist_path
46             )
47             logger.debug(f"Initialized vectorstore for {provider} at {persist_path}")
48         logger.info(f"Vectorstore initialization complete.")
49
50 async def upload_vectorstore_from_pdf(upload_files: List[UploadFile], model_provider: str):
51     logger.debug(f"Uploading vectorstore for {model_provider}")
52     file_paths = await save_uploaded_files(upload_files)
53     docs = load_documents_from_paths(file_paths)
54     chunks = split_documents_to_chunks(docs)
55     embedding = get_embeddings(model_provider)
56     persist_path = VECTORS_STORE_DIRECTORY[model_provider]
57
58     if vectorstore_exists(persist_path):
59         logger.debug(f"Appending to existing vectorstore...")
60         vectorstore = Chroma(persist_directory=persist_path, embedding_function=embedding)
61         vectorstore.add_documents(chunks)
62         logger.debug(f"Added {len(chunks)} chunks to existing vectorstore.")
63     else:
64         vectorstore = Chroma.from_documents(docs, embedding=embedding, persist_directory=persist_path)
65         logger.debug(f"Created new vectorstore with {len(chunks)} chunks.")
66
67     return vectorstore
68
69 def load_vectorstore(model_provider: str):
70     persist_path = VECTORS_STORE_DIRECTORY[model_provider]
71     logger.debug(f>Loading vectorstore from {persist_path}")
72
73     if vectorstore_exists(persist_path):
74         logger.debug(f>Loading existing vectorstore for provider: {model_provider}")
75         return Chroma(persist_directory=persist_path, embedding_function=get_embeddings(model_provider))
76     else:
77         logger.error(f"Vectorstore not found for provider: {model_provider}")
78         raise ValueError(f"Vectorstore not found for provider: {model_provider}")
79
80 def get_collection_count(model_provider: str):
81     logger.debug(f>Getting collection count for provider: {model_provider}")
82     vectorstore = load_vectorstore(model_provider)
83     return vectorstore.collection.count()
84
85 def find_similar_chunks(model_provider: str, query: str):
86     logger.debug(f>Searching for similar chunks for provider: {model_provider}")
87     vectorstore = load_vectorstore(model_provider)
88     return vectorstore.similarity_search(query)
```

Figure A. 3 Embedding Generation and Similarity-Based Retrieval Code

A.4 RAG Answer Generation Code

This code demonstrates how the retrieved document context is combined with the user's question and passed to the language model.

The system follows the sequence:

User Question → Retrieved Context → Prompt → Language Model → Generated Answer

The main application supports configurable language-model providers, while the final generation experiment used the locally executed Qwen2.5-3B model through Ollama..

```
llm_chain_factory.py M X
server > core > llm_chain_factory.py > ...

1 from server.config.settings import GROQ_API_KEY, GOOGLE_API_KEY
2 from langchain_core.prompts import ChatPromptTemplate
3 from langchain_classic.chains import create_retrieval_chain
4 from langchain_classic.chains.combine_documents import create_stuff_documents_chain
5
6 from langchain_google_genai import ChatGoogleGenerativeAI
7 from langchain_groq import ChatGrog
8
9 from server.utils.logger import logger
10
11 def get_prompt():
12     logger.debug("Creating chat prompt template.")
13     return ChatPromptTemplate.from_messages([
14         ("system", "Answer as detailed as possible using the context below. If unknown, say 'I don't know.'"),
15         ("human", "Context:\n{context}\n\nQuestion:\n{input}")
16     ])
17
18 def get_llm(model_provider: str, model: str):
19     logger.debug(f"Initializing LLM for {model_provider} - {model}")
20     if model_provider == "grog":
21         return ChatGrog(model=model, api_key=GROQ_API_KEY)
22     elif model_provider == "gemini":
23         return ChatGoogleGenerativeAI(model=model, api_key=GOOGLE_API_KEY)
24     else:
25         logger.error(f"Unsupported LLM Provider: {model_provider}")
26         raise ValueError(f"Unsupported LLM Provider: {model_provider}")
27
28 def build_llm_chain(model_provider: str, model: str, vectorstore):
29     logger.debug(f"Building LLM chain for provider: {model_provider}, model: {model}")
30     prompt = get_prompt()
31     llm = get_llm(model_provider, model)
32     retriever = vectorstore.as_retriever(search_kwargs={"k": 6})
33
34     return create_retrieval_chain(
35         retriever,
36         create_stuff_documents_chain(llm, prompt=prompt)
37     )
38
You, 59 minutes ago • Uncommitted changes
```

Figure A. 4: RAG Answer Generation Code

A.5 Evaluation and Result Analysis Code

This section contains the code used to evaluate the RAG system. Retrieval performance is evaluated using **Hit@k**, **MRR**, and **nDCG**, while generation performance is evaluated using **lexical F1**, **IDK percentage**, **latency**, and **context size**.

The generation experiment compares:

- **R0_k3**
- **R1_k10**
- **R2_k10_optimized**

The resulting evaluation tables are generated for overall performance, question type, source type, and paired lexical-F1 differences.

```

run_generation_experiment.py | run_generation_experiment.py | score_generation.py
benchmarks | open_vagbench | run_generation_experiment.py | benchmarks | open_vagbench | score_generation.py

1 import json
2 import os
3 import time
4 from pathlib import Path
5
6 import numpy as np
7 import pandas as pd
8 from dotenv import load_dotenv
9 from langchain_ollama import ChatOllama
10 from sentence_transformers import SentenceTransformer
11
12 BASE_DIR = Path(__file__).resolve().parent
13 PROJECT_DIR = BASE_DIR.parent.parent
14
15 QUESTIONS_FILE = BASE_DIR / "generation_eval_60.csv"
16 EMBEDDINGS_FILE = BASE_DIR / "corpus_embeddings.npy"
17 METADATA_FILE = BASE_DIR / "corpus_metadata.json"
18
19 OUTPUT_DIR = BASE_DIR / "generation_results"
20 OUTPUT.mkdir(exist_ok=True)
21
22 MODEL_NAME = "qwen2.5:72b"
23 EMBEDDING_DIM = 384
24 OPTIMIZED_CONTEXT_SIZE = 5
25 MAX_CONTEXT_CHARACTERS = 8000
26 MAX_OUTPUT_TOKENS = 512
27
28 ENV_FILE = PROJECT_DIR / ".env"
29 load_dotenv(ENV_FILE)
30
31 print("\n\n")
32 print("FINAL RAG GENERATION EXPERIMENT")
33 print("\n\n")
34 print(f"Model: {MODEL_NAME}")
35
36 llm = ChatOllama(
37     model=MODEL_NAME,
38     temperature=0,
39     max_tokens=MAX_OUTPUT_TOKENS,
40 )
41
42 print("\nLoading generation subset...")
43 questions = pd.read_csv(QUESTIONS_FILE)
44 print(f"Questions loaded: {len(questions)}")
45
46 print("\nLoading corpus embeddings...")
47
48 if embeddings.shape[1] != EMBEDDING_DIM:
49     raise RuntimeError(
50         f"Expected embedding dimension {EMBEDDING_DIM}, "
51         f"got {embeddings.shape[1]}"
52     )
53
54 print("\nNormalizing embeddings...")
55
56 embedding_norms = np.linalg.norm(
57     embeddings,
58     axis=1,
59     keepdims=True
60 )
61
62 embedding_norms[embedding_norms == 0] = 1
63 normalized_embeddings = embeddings / embedding_norms
64
65 print("\nLoading query embedding model...")
66
67 embedding_model = SentenceTransformer(
68     "sentence-transformers/all-MiniLM-L12-V2"
69 )
70
71 > def retrieve(question, k):...
72
73 > def optimize_context(retrieved, max_sections=5):...
74
75 > def limit_context_characters(context_items, max_characters=MAX_CONTEXT_CHARACTERS):...
76
77 > SYSTEM_PROMPT = """...
78
79 > def build_prompt(question, context_items):...
80
81 > def generate_answer(question, context_items):...
82
83 > def run_condition(condition_name, k, optimize=False):...
84
85 > def main():...
86
87 if __name__ == "__main__":
88     main()
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2
```